

Руководство внедренца

Med Doctor Assignment Service. Версия документации: 2026-04-29.

Содержание

Руководство внедренца

1. Цель внедрения
2. Роли и зоны ответственности
3. Что нужно получить от заказчика
4. Проверка готовности Orchestra
5. Безопасный порядок внедрения
 - Этап 1. Подготовка конфигурации
 - Этап 2. Первый запуск без мутаций
 - Этап 3. Проверка websocket
 - Этап 4. Проверка med-robot
 - Этап 5. Ограниченная реальная проверка
 - Этап 6. Расширение rollout
6. Проверка REST-контрактов workflow
7. Настройки, которые нельзя включать без причины
8. Приемочные сценарии
9. Документы, которые передаются заказчику
10. Финальный чек-лист внедренца

1. Цель внедрения

Med Doctor Assignment Service внедряется для автоматического назначения визитов из очереди «Врач не назначен» на врача, который вошел в рабочее место и получил рабочий профиль. Сервис может выбирать следующую услугу локально по данным Orchestra или обращаться к med-robot, который возвращает оптимальные `serviceId` и `queueId`.

Задача внедренца - безопасно подключить сервис к конкретной инсталляции Orchestra, подтвердить REST-контракты, настроить id отделений/очередей/entry point, выполнить тесты на стенде и только затем включить реальные мутации.

2. Роли и зоны ответственности

Роль	Ответственность
Внедренец	сбор исходных id, настройка <code>application.yml</code> , dry-run проверка, первичный rollout.
Администратор Orchestra	учетная запись, права, справочники, тестовые визиты, рабочие профили.
Разработчик	исправление кода/контрактов при несовпадении REST API конкретной Orchestra.
Техподдержка	анализ логов, контроль восстановления после сбоев.
Тестировщик	функциональные сценарии, негативные проверки, приемочный протокол.

3. Что нужно получить от заказчика

Данные	Пример	Комментарий
URL Orchestra	<code>http://192.168.7.135:8080</code>	без завершающего слэша.
Логин/пароль сервисной учетной записи	<code>superadmin / ***</code>	лучше отдельная учетная запись, не личный пользователь.
Branch id тестового отделения	<code>1</code>	используется в <code>branches-for-cache</code> и <code>allowed-branches</code> .
Queue id «Врач не назначен»	<code>292</code>	это queue id, не service id.
Source entry point id	<code>1</code>	используется как <code>fromId</code> при transfer.
Точка обслуживания врача	<code>servicePointId=...</code>	нужна для проверки событий посадки.
Staff id тестового врача	<code>staffId=...</code>	нужен для диагностики.
Work profile врача	<code>workProfileId=...</code>	должен содержать ожидаемые очереди/услуги.
Тестовые услуги	<code>serviceId=...</code>	должны присутствовать в маршрутах тестовых визитов.
URL med-robot	<code>http://...:8082</code>	если используется внешний выбор.

4. Проверка готовности Orchestra

Перед запуском сервиса проверьте:

- сервисная учетная запись авторизуется в REST Orchestra;
- доступны справочники отделений, очередей, услуг, точек обслуживания и рабочих профилей;
- в тестовом отделении есть очередь «Врач не назначен»;
- рабочий профиль тестового врача связан с нужными очередями;
- тестовый визит находится в очереди «Врач не назначен»;
- при посадке врача Orchestra публикует `USER_SERVICE_POINT_SESSION_START` и `SET_WORK_PROFILE`;
- endpoint перевода визита ожидает корректный `fromId`, а не случайный id очереди.

5. Безопасный порядок внедрения

Этап 1. Подготовка конфигурации

Создайте внешний файл `application-test.yml`. Не правьте jar и не храните пароль в git.

Минимальные безопасные настройки:

```
application:
  orchestra:
    url: http://orchestra-test.local:8080
    username: ${ORCHESTRA_USER:superadmin}
    password: ${ORCHESTRA_PASSWORD:change-me}
    branches-for-cache: "1"
    replay-mutation-cookies: false
    reconnect-delay-ms: 30000

  websocket:
    enabled: false

  med-robot:
    enabled: false

  assignment:
    enabled: true
    unknown-doctor-queue-id: 292
    dry-run: true
    polling-enabled: true
    polling-cron: "0 */1 * * * ?"
    max-visits-per-cycle: 3
    allowed-branches: [1]
    recheck-visit-before-transfer: true
    abort-cycle-on-forbidden-mutation: true
    source-entry-point-id-by-branch:
      1: 1
```

Этап 2. Первый запуск без мутаций

Ожидаемые признаки успеха:

- сервис стартовал и не завершился аварийно;
- есть `Start cache refresh for branch`;
- есть `Finish cache refresh for branch`;
- polling видит очередь «Врач не назначен»;
- при наличии визитов сервис пишет план действий, но не выполняет реальные PUT/POST.

Этап 3. Проверка websocket

Включите websocket, но оставьте `dry-run=true`.

```
application:
  websocket:
    enabled: true
    subscribed-events:
      - USER_SERVICE_POINT_SESSION_START
      - SET_WORK_PROFILE
  assignment:
    dry-run: true
    service-point-open-trigger-enabled: false
    set-work-profile-trigger-enabled: false
    user-service-point-session-start-trigger-enabled: true
```

Посадите тестового врача на рабочее место. В логах должна появиться корреляция сессии и рабочего профиля. Сырые `SERVICE_POINT_OPEN` и `SET_WORK_PROFILE` не должны сами запускать опасные мутации.

Этап 4. Проверка med-robot

Если заказчик использует med-robot, включите его на dry-run.

```
application:
  med-robot:
    enabled: true
    url: http://med-robot-test.local:8082
    request-body-mode: TICKET_NUMBER_PLAIN_TEXT
    error-handling-mode: FALLBACK_TO_LOCAL
    fallback-to-local-on-empty-response: true
    require-known-queue: true
```

При пустом `unservedVisitServices` основной режим - `TICKET_NUMBER_PLAIN_TEXT`: сервис передает номер талона в body `text/plain`.

Этап 5. Ограниченная реальная проверка

Только после успешных dry-run проверок:

```
application:
  assignment:
    dry-run: false
    max-visits-per-cycle: 1
    allowed-branches: [1]
```

Выполните один тестовый сценарий с одним визитом. После проверки результата верните `dry-run=true`, если нужно анализировать логи до продолжения.

Этап 6. Расширение rollout

Расширяйте `allowed-branches` постепенно. Не используйте пустой список `allowed-branches` до завершения приемки на всех отделениях.

6. Проверка REST-контрактов workflow

Сервис использует конфигулируемые пути:

```
application:
  assignment:
    experimental-endpoints:
      enabled: true
      queue-visits-path: "/rest/entrypoint/branches/{branchId}/queues/{queueId}/visits/
full/"
      visit-details-path: "/rest/entrypoint/branches/{branchId}/visits/{visitId}/"
      visit-by-id-path: "/rest/entrypoint/branches/{branchId}/visits/{visitId}/"
      assign-service-path: "/rest/entrypoint/branches/{branchId}/visits/{visitId}/services/
{serviceId}/"
      transfer-visit-path: "/rest/entrypoint/branches/{branchId}/queues/{queueId}/visits/"
```

Что подтвердить на конкретной Orchestra:

- `queue-visits-path` возвращает список визитов очереди;
- `visit-details-path` содержит `ticketId`, `currentVisitService`, `currentService`, `unservedVisitServices`;
- `assign-service-path` реально назначает услугу или возвращает состояние, достаточное для `transfer-only`;
- `transfer-visit-path` принимает payload с `fromBranchId`, `fromId`, `visitId`;
- `fromId` соответствует source entry point id, а не случайному id очереди.

7. Настройки, которые нельзя включать без причины

Настройка	Почему осторожно
<code>service-point-open-trigger-enabled=true</code>	событие может быть преждевременным, профиль врача еще не стабилизирован.
<code>set-work-profile-trigger-enabled=true</code>	сырой профиль без сессии может привести к неверному контексту.
<code>dry-run=false</code> на всех отделениях	риск массовых мутаций при ошибке id.
<code>allowed-branches: []</code>	означает все отделения.
<code>replay-mutation-cookies=true</code>	по наблюдениям может провоцировать повторные 403.
<code>send-cookies-in-handshake=true</code>	может смешать REST-cookie и websocket handshake.
<code>abort-cycle-on-forbidden-mutation=false</code>	при ошибке контекста сервис продолжит бить Orchestra одинаковыми запросами.

8. Приемочные сценарии

Сценарий	Ожидаемый результат
Старт при доступной Orchestra	cache refresh завершен, polling работает.
Старт при недоступной Orchestra	сервис не падает, планирует retry.
Посадка врача	корреляция USER_SERVICE_POINT_SESSION_START -> SET_WORK_PROFILE.
Визит в «Врач не назначен»	сервис видит визит в очереди.
Dry-run назначение	лог содержит выбранную услугу/очередь без мутаций.
med-robot plain text	body содержит номер талона, ответ принят.
Ошибка med-robot	fallback или skip согласно настройке.
Реальное назначение одного визита	визит переведен в ожидаемую очередь.
Повторный маршрутный шаг	новый currentVisitService.id допускает повторную обработку того же visitId.
Первый 403 на mutation	цикл остановлен, нет шквала запросов.

9. Документы, которые передаются заказчику

- README.md - общее описание и архитектура.
- docs/TEST_STAND_SETUP.md - настройка тестового стенда.
- CONFIGURATION_REFERENCE.md - справочник параметров.
- OPERATION_MODES.md - режимы работы.
- RUNBOOK.md - сопровождение и диагностика.
- MED_ROBOT_INTEGRATION.md - интеграция с med-robot.
- ORCHESTRA_REST_CONTRACTS.md - REST-контракты Orchestra.

10. Финальный чек-лист внедренца

- [] Подтверждены branch id и queue id.
- [] Подтвержден source entry point id для transfer.
- [] Настроен внешний application-test.yml.
- [] Первый запуск выполнен с dry-run=true.
- [] Проверен refresh branch cache.
- [] Проверена очередь «Врач не назначен».
- [] Проверена посадка врача и рабочий профиль.
- [] Проверен med-robot, если он нужен.
- [] Реальная проверка выполнена с max-visits-per-cycle=1.

- [] Логи успешного сценария сохранены.
- [] Согласован порядок расширения `allowed-branches`.